

Herald



Protocol Research: OpenAI Realtime API

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	OpenAI Realtime API is a WebSocket-based protocol for fully-asynchronous, multimodal (text + audio) interaction with OpenAI's models. ~20 distinct event types; PCM16 / G.711 audio formats. For Herald, this is NOT an ingest or fan-out protocol — it's an OUTBOUND DISPATCHER (Herald sends notification content TO an OpenAI-hosted model for transformation/synthesis, e.g. text-to-speech voice alerts). Joins <code>commons_messaging/dispatch/openai_realtime/.P2</code> (opt-in).
Issues	open-questions: voice alerts a Herald MVP feature?; cost ceiling; audio output format selection.
Continuation	Wave 4f+ — open HRD-311..HRD-318 IF voice alerts become a Herald feature. Else defer indefinitely.

Constitutional anchors

- **§107 anti-bluff** — tests require a real OpenAI API key + real connection to `wss://api.openai.com/v1/realtime`; assert audio bytes received + transcript matches expected.
- **§11.4.74 catalogue-check** — no `vasic-digital` or `HelixDevelopment` OpenAI Realtime module. Use OpenAI's official Go SDK if available; else hand-roll WS client.
- **§11.4.61** — tracked doc.

Table of contents

- [§1. Protocol overview](#)
- [§2. Specification deep-dive](#)
- [§3. Herald-specific applicability analysis](#)
- [§4. Step-by-step implementation guide for Herald](#)
- [§5. §107 anti-bluff testing strategy](#)
- [§6. Open questions for operator](#)

- [§7. References](#)
- [§8. Catalogue-check verdict](#)

§1. Protocol overview

OpenAI Realtime API — a WebSocket-based protocol introduced by OpenAI in late 2024 for low-latency, multimodal interactions with GPT models. Specifically designed for voice agents (real-time speech-to-text + LLM + text-to-speech round-tripping). License: proprietary (OpenAI-hosted; clients use OpenAI SDKs or hand-roll).

Wire format. JSON events over a single WebSocket. ~20 distinct event types (server-to-client and client-to-server). Connection: `wss://api.openai.com/v1/realtime?model=gpt-4o-realtime-preview`.

Authentication. OpenAI API key in HTTP Authorization: Bearer <key> header on the WS upgrade. Optional OpenAI-Safety-Identifier header for end-user attribution.

Modalities. Text + audio (input: PCM16 / G.711 μ -law / G.711 A-law; output: same set). Function calling supported.

Session model. The Realtime session maintains conversation context server-side — no need to resend history each turn. Session events: `session.created`, `session.update`, `session.error`.

Adoption. Used by voice-AI startups, customer-support assistants, OpenAI's "Advanced Voice Mode" in ChatGPT, Microsoft Foundry's GPT-Realtime equivalents.

§2. Specification deep-dive

§2.1 Connection

```
wss://api.openai.com/v1/realtime?model=gpt-4o-realtime-preview
Authorization: Bearer sk-...
OpenAI-Safety-Identifier: <opaque-user-id>
```

§2.2 Event types (client-to-server)

- `session.update` — update session config (instructions, voice, audio format).
- `input_audio_buffer.append` — append base64 PCM audio chunk.
- `input_audio_buffer.commit` — commit current buffer as one user turn.
- `conversation.item.create` — add a text message.
- `response.create` — request the model to respond.
- `response.cancel` — cancel an in-progress response.

§2.3 Event types (server-to-client)

- `session.created` — initial session ack.
- `conversation.item.created` — server logged an item.
- `response.created` — model started responding.
- `response.audio.delta` — incremental audio chunk (base64 PCM).
- `response.audio.done` — audio complete.
- `response.text.delta` — incremental text chunk.

- `response.text.done` — text complete.
- `response.function_call_arguments.delta` — incremental function args.
- `response.done` — full turn complete.
- `error` — any error.

§2.4 Audio formats

- **PCM16** — 16-bit PCM at 24 kHz. Highest quality. Default.
- **G.711 μ -law** — 8-bit μ -law at 8 kHz. Telephony.
- **G.711 A-law** — 8-bit A-law at 8 kHz. European telephony.

§2.5 Function calling

Server can emit `response.function_call_arguments.delta` indicating the model wants to call a function. Client executes the function, then sends `conversation.item.create` with the function result, then `response.create` for the next turn.

§2.6 Pricing

OpenAI charges per audio-token-equivalent. ~\$0.06 per minute of audio output (subject to change). Realtime is meaningfully more expensive than text-only LLM calls.

§3. Herald-specific applicability analysis

§3.1 The Herald use case for Realtime

Herald is an event fan-out + notification system. The intersection with OpenAI Realtime is:

- **Voice-notification dispatch** — Herald takes a notification text ("P1 incident #4321: database down") and uses OpenAI Realtime to synthesize SPOKEN audio that's then sent via a phone-call channel (Twilio Voice integration, future) or attached as an .mp3 to a Telegram/Slack message.
- **Voice-incident chat** — iherald's incident war room could accept VOICE input from on-call engineers via WebRTC → relay to Realtime → return spoken AI summary. (This crosses into WebRTC territory, which is REJECTED per README.md §5 as out of scope.)

The realistic Herald use case is the FIRST: text → speech for voice-channel delivery.

§3.2 Why this is a DISPATCHER, not an ingest protocol

Realtime is a CLIENT relationship from Herald to OpenAI. Herald CONSUMES the API; it doesn't expose anything. The dispatch surface lives at `commons_messaging/dispatch/openai_realtime/`, analogous to the existing `commons_messaging/dispatch/claude_code/`.

§3.3 Cost model concerns

A single voice notification is ~5-10 seconds of audio, ~0.005–0.01 per dispatch. Per-tenant rate limiting MUST be enforced; budget check against bherald.

§3.4 Auth + tenant model

OpenAI API key is GLOBAL per Herald deployment (one key serves all tenants), but cost MUST be attributed per-tenant. Use `OpenAI-Safety-Identifier: tenant:<tenant_id>` for OpenAI's audit logs; cross-reference Herald's outbound dispatch records.

§3.5 Failure modes

1. **OpenAI outage.** Fallback: store the text + retry; eventually fall back to a non-voice channel.
2. **Cost runaway.** Hard ceiling per-tenant per-day (configured via bherald).
3. **API key rotation.** Standard rotation via Vault / Doppler / OPS pattern.
4. **Audio quality complaints.** Test with a representative sample; document voice + audio_format choice.

§4. Step-by-step implementation guide for Herald

§4.1 Add dep

If OpenAI ships an official Go SDK for Realtime (as of 2026-05-22 partial support in `github.com/openai/openai-go`), use it. Else hand-roll a thin WS client on top of `coder/websocket`.

§4.2 New dispatcher `commons_messaging/dispatch/openai_realtime/`

```
commons_messaging/dispatch/openai_realtime/
├── client.go           # WS connection management
├── session.go          # session.update + event dispatch
├── tts.go              # text-to-speech wrapper (high-level API)
├── budget.go           # per-tenant cost tracking
└── client_test.go
```

§4.3 Per-channel integration

The OpenAI Realtime dispatcher is consumed by:

- A new `commons_messaging/channels/voice_telegram/` — uses Realtime to synthesize audio + attaches to Telegram message via tgram channel.
- A future `commons_messaging/channels/voice_twilio/` — uses Realtime + Twilio to place phone calls (WAY out of scope for MVP).

§4.4 Budget integration

Every Realtime invocation logs to `events_processed` with a cost metadata field. bherald aggregates; alerts on overruns.

§4.5 New `e2e_bluff_hunt` invariants

- **E90** — real OpenAI connect: `pherald_notify --voice` sends a notification; assert audio bytes received from Realtime + saved to `/tmp` + `duration > 0`.

- **E91** — fallback: simulate OpenAI 503; assert dispatch falls back to plain-text channel.
- **E92** — budget cap: enqueue voice notifications until ceiling; assert subsequent dispatches reject with 429.
- **E93** — safety identifier: assert `OpenAI-Safety-Identifier: tenant:<id>` set on every connect.

(E90 is OPTIONAL — runs only with `OPENAI_API_KEY` set; skipped in CI by default.)

§4.6 HRD scaffolding

- **HRD-311** — bootstrap `commons_messaging/dispatch/openai_realtime/`.
- **HRD-312** — WS client + session management.
- **HRD-313** — TTS high-level API.
- **HRD-314** — budget tracking + per-tenant rate limit.
- **HRD-315** — fallback on outage.
- **HRD-316** — channel adapter (`voice_telegram`).
- **HRD-317** — `e2e_bluff_hunt` E90–E93 + spec amendments + operator credentials guide update.
- **HRD-318** — close Wave 4f.

§5. §107 anti-bluff testing strategy

The bar: Herald connects to `real wss://api.openai.com/v1/realtime`, sends text, receives audio, plays audio (or saves to file), audio is intelligible.

§5.1 Happy-path (requires API key)

- **Test 1: text → audio.** Connect; send `conversation.item.create` with text "Hello world"; receive `response.audio.delta` events; concatenate; save as `out.wav`.
Physical proof: `out.wav` file exists, has nonzero size, and `ffprobe` reports valid PCM 16-bit / 24kHz.
- **Test 2: function-call round trip.** Configure a function `get_incident_status`; send query; receive `response.function_call_arguments.done`; respond with result; receive final answer.

§5.2 Edge cases (mock the WS server)

A local mock WS server stands in for OpenAI's API for fast CI tests. The mock responds with canned events for known prompts. Tests:

- **Test 3: session lifecycle.** Connect → `session.created`. Send `session.update` → assert echo. Send `response.create` → assert `response.done`.
- **Test 4: cancellation.** Mid-response, send `response.cancel` → assert `response.done` with cancellation status.
- **Test 5: error event.** Mock emits error; Herald handles + logs + reconnects.

§5.3 Failure modes

- **Test 6: bad API key.** WS upgrade returns 401.
- **Test 7: rate limit.** Mock returns 429; Herald backs off.
- **Test 8: connection drop mid-audio.** Mock closes mid-stream; Herald saves partial; reconnects + retries.

§5.4 Cost + budget

- **Test 9: budget cap.** Configure cap; dispatch until cap; assert subsequent dispatches FAIL gracefully + emit OTel alert.

§5.5 Mutation gates

- Mutation: remove `OpenAI-Safety-Identifier` header → Test 1 still works (OpenAI doesn't fail) but audit linkage breaks; PHYSICAL TEST: query OpenAI audit log via OpenAI's logs API; assert `tenant_id` absent → mutation gate FAILS as expected.

§5.6 Wire-level

- tcpdump (with TLS key log to decrypt) the WS frames → verify JSON event types match spec.

§5.7 Audio-fidelity assertion

`out.wav` from Test 1 is sent through `whisper-1` (or equivalent local model) for transcription → assert transcript $\approx$ original input (BLEU > 0.9 or simple substring match).

§6. Open questions for operator

1. **Voice alerts an MVP feature?** Recommend NO for Wave 4; revisit Wave 5+ if customer demand.
2. **OpenAI as the only voice provider?** Anthropic, Google, Microsoft have similar APIs. Recommend OpenAI first (most mature); add others later.
3. **Audio format default?** Recommend PCM16 — highest quality.
4. **Voice selection?** OpenAI has 10+ voices; tenant config.
5. **Cost ceiling per tenant per day?** Recommend \$5/day default.
6. **Mock-only CI or real-API CI?** Recommend mock for CI; real-API for nightly + pre-release.

§7. References

(All fetched 2026-05-22.)

- **Realtime API docs.** <https://developers.openai.com/api/docs/guides/realtime>
- **WebSocket mode.** <https://developers.openai.com/api/docs/guides/realtime-websocket>
- **Microsoft Foundry equivalent.** <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/realtime-audio-websockets>
- **OpenAI Realtime playground.** <https://platform.openai.com/playground/realtime>
- **OpenAI Go SDK.** <https://github.com/openai/openai-go>
- **Forasoft 2026 integration guide.** <https://www.forasoft.com/blog/article/openai-realtime-api-webrtc-sip-websockets-integration>

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital:** no module.

- **HelixDevelopment:** no module.

Verdict: no-match. Depend on `github.com/openai/openai-go` if it supports Realtime (check at implementation time); else hand-roll on top of `commons_ws/ client`. No new submodule needed.